

Deploy your apps to a supercloud in a few clicks

This Engineering Education program is supported by Section. Instantly deploy your GitHub apps, Docker containers or K8s namespaces to a supercloud.

[Try It For Free](#)

Basics of SSH and Building an Application in Node.js

September 29, 2020

Topics: [Node.js](#)



Accessing systems remotely from your home computer is crucial for software developers and system administrators. Users might need to configure, install, and run applications on servers and computers that might not be physically accessible to them. Nowadays, cloud computing is one of the most popular platforms for hosting and running applications over the internet.

In the cloud ecosystem, users create virtual computers called [virtual machines](#) that run in data centers somewhere in the world. Users require

SSH is one such tool that allows users to securely connect to and control remote computers from the comfort of their homes. In this article, we are going to explore the basics of SSH and build an application in Node.js to connect to a virtual machine and monitor system health.

What is SSH?

SSH stands for **Secure Shell** (also called **Secure Socket Shell**) and is a protocol that allows users to access a remote system or a virtual machine using the command line or the terminal. SSH establishes a shell session with the remote host, and the user will be able to manage the remote host by executing commands on their local computer via a terminal or command-line interface.

In other words, users can “log in” to a virtual machine and manipulate it based on their requirements. SSH is a very powerful tool for IT professionals and system administrators who deal with configuring and managing servers on a daily basis. Without SSH, users would have to physically go to remote data centers and connect to the computers, which is not feasible or efficient.

SSH is based on the **client-server** model, where a client is an application on the local system, and the server is the host you wish to connect to. The client application takes the remote host information, such as username, password, etc., as input and establishes an encrypted SSH session with the remote host.

...are correct, the request is rejected, and an SSH connection is not established.

Using SSH

Most UNIX based operating systems such as Ubuntu, MAC OS, etc. come with an SSH client pre-installed. To establish a connection, open the terminal and execute the following command:

```
ssh <username>@<hostname or host ip>
```

The username field specifies the user id of the machine you are connecting to. The hostname is the name of the remote system and can be found by typing `hostname` in the terminal. Users can also specify the host IP instead of the hostname. for example:

```
ssh adith@219.123.101.201
```

This establishes a connection with the host whose IP is: 219.123.101.201 and the user is adith.

SSH in Node.js

We are going to build an application that connects to a virtual machine and extracts information on the filesystem health of the VM. The application monitors the disk utilization in the VM and flags the partitions that have a disk utilization greater than 90%. This is useful for system administrators as it tells which partitions need to be monitored and

Node.js has an array of libraries for establishing an SSH connection. In this article, we are going to use [simple-ssh](#). Simple-ssh is a robust and easy to use library that allows us to connect to a VM and execute Linux commands using Node.js.

To install and setup Node.js, follow [this tutorial](#)

Install the simple-ssh library by running the following command:

```
npm install simple-ssh
```

Install the node-csv-parse library(to parse the output of the `df -h` command) by running the following command:

```
npm install node-csv-parse
```

Code

```
// required to take input from user
const readline = require("readline");
const rl = readline.createInterface({
  input: process.stdin,
  output: process.stdout
});

const SSH = require('simple-ssh');
```

The `readline` library allows users to provide input using the command line. We are going to take the VM host details such as hostname, username, and password as input from the user. This is required to establish a secure connection with the remote host.

```
// function to get input from user
function getInput() {

    rl.question("What is the host? ", function(host) {
        rl.question("What is the user? ", function(user) {
            rl.question("What is the password? ", function(password) {

                ssh(host, user ,password);
                rl.close();

            });
        });
    });
}
```

The `getInput()` function takes input from the user. The hostname, username, and password of the VM are stored in the `host`, `user`, and `password` variables and are passed to the `ssh()` function as parameters.

```
// function to ssh into a remote host.
function ssh(host, user ,password){

    console.log('inside the system')
```

```
        pass: password
    });
    // execute the df -h command to find out disk utilization
    ssh.exec('df -h', {
        out: function(stdout) {
            parse(stdout);
        }
    }).start();
}
```

The `ssh()` function takes the hostname, username, and password as the input and establishes an SSH connection with the VM using `simple-ssh`. The `exec()` function executes a Linux command on the remote host. In this case, we execute the `df -h` command that shows us the amount of disk space that is free on the host.

Once we get the output of the `df -h` command in the `stdout` variable, we pass it to the `parse()` function that parses the output and displays it in a presentable format.

```

/dev/sda10      114G   69G   39G   64% /
tmpfs           3.9G  184M   3.7G    5% /dev/shm
tmpfs           5.0M   4.0K   5.0M    1% /run/lock
tmpfs           3.9G    0    3.9G    0% /sys/fs/cgroup
/dev/loop0      256M  256M    0 100% /snap/gnome-3-34-1804/33
/dev/loop8       68M   68M    0 100% /snap/sublime-text/85
/dev/loop3       2.5M   2.5M    0 100% /snap/gnome-calculator/730
/dev/loop5       141M  141M    0 100% /snap/gnome-3-26-1604/100
/dev/loop7       68M   68M    0 100% /snap/sublime-text/77
/dev/loop9       55M   55M    0 100% /snap/gtk-common-themes/1502
/dev/loop10      141M  141M    0 100% /snap/gnome-3-26-1604/98
/dev/loop1       256M  256M    0 100% /snap/gnome-3-34-1804/36
/dev/loop4       2.5M   2.5M    0 100% /snap/gnome-calculator/748
/dev/loop6       55M   55M    0 100% /snap/core18/1880
/dev/loop11      2.3M   2.3M    0 100% /snap/gnome-system-monitor/145
/dev/loop12      164M  164M    0 100% /snap/spotify/41
/dev/loop13      162M  162M    0 100% /snap/gnome-3-28-1804/128
/dev/loop14       56M   56M    0 100% /snap/core18/1885
/dev/loop15      97M   97M    0 100% /snap/core/9804
/dev/loop16      161M  161M    0 100% /snap/gnome-3-28-1804/116
/dev/loop17      384K  384K    0 100% /snap/gnome-characters/550
/dev/loop20      384K  384K    0 100% /snap/gnome-characters/539
/dev/loop18       63M   63M    0 100% /snap/gtk-common-themes/1506
/dev/loop19      1.0M   1.0M    0 100% /snap/gnome-logs/93
/dev/loop21      2.3M   2.3M    0 100% /snap/gnome-system-monitor/148
/dev/loop22      1.0M   1.0M    0 100% /snap/gnome-logs/100
/dev/sda1       496M   78M  419M   16% /boot/efi
tmpfs           785M   20K   785M    1% /run/user/120
tmpfs           785M   60K   785M    1% /run/user/1000
/dev/loop23      98M   98M    0 100% /snap/core/9993
/dev/sda5       244G  156G   89G   64% /media/adith/New Volume
adith@lawliet:~/engineering-education/articles/ssh-in-nodejs$

```

The output of the `df` command is hard to read and understand. Therefore, we need to parse it and extract the relevant data.

```

// Threshold value to see if disk utilization is below or above the desired
const threshold = 90;
// function to parse the raw data from the df -h command.
function parse(data){

```

```
// The trim value specifies whether the extra spaces in the data need
var parsed = parser(data, {
    delimiter: ' ',
    trim: true
}).asRows();

var finalData = [];

// go through every row in the data and add another column for threshold
// Also, check to see if CPU utilization percentage has exceeded threshold
for(var i = 0; i < parsed.length; i++){
    // Split the data based on space so that we get the individual columns
    var temp = parsed[i][0].split(' ');

    // remove all the empty spaces in the array
    temp = temp.filter(function(str) {
        // This line removes all the empty spaces in the array
        return /\S/.test(str);
    });

    // If this is the first record, then it is the column header. Then
    // called Threshold exceeded? which tells us whether the limit has
    if(i == 0){
        temp.pop();
        temp.push('Threshold exceeded?');
    }

    // If this is not the first row, then we can check to see if it has
    else{
        if(parseInt(temp[4]) >= threshold){
            temp.push('YES');
        }
        else{
            temp.push('NO');
        }
    }
}
```

```
// Print the final data as a neat table on the terminal
console.table(finalData);
}
```

In the `parse()` function, we go through every row in the data and see if the CPU utilization percentage has exceeded the threshold or not. For this, we add another column to the data called `Threshold exceeded?`. This column has two values, “yes” or “no” and tells the user whether the threshold was exceeded or not.

```
function main(){
    getInput();
}

main();
```

Conclusion and Further reading

To recap, SSH is a protocol that allows users to access a remote system, and is a very useful tool for developers and system administrators. We explored how SSH works and how to use SSH in your local system using the terminal. We understood how we can leverage the power of Node.js to write an application that uses `simple-ssh` (A library that allows users to use SSH in Node.js) to SSH into a virtual machine, monitor the system disk usage, and display it to the user in a presentable format. Now that you know how SSH works and how to use it in Node.js, you can write your own application to do much more.

Did you find this article helpful?



3 Comments

Sort By Best ▼

The EngEd community is subject to Section's [moderation policy](#).

Write your comment...

[LOGIN](#) [SIGNUP](#)

a

ahmet can 1 year ago

i get error Uncaught Error: Cannot find module 'net'. How can i solve this problem?

[Reply](#) [Share](#)

 0  0

J

Jorge 1 year ago

This is the error I'm getting TypeError: ssh.exec is not a function

[Reply](#) [Share](#)

 0  0

F

Facu 1 year ago

this is because you are using the 'ssh' variable, you need to change 'ssh.exec' to 'ssh_options.exec'. That's a small bug in the code.

[Reply](#) [Share](#)

 0  0

Similar Articles



Languages, Node.js

Building a Payroll System with Next.js

[Read More →](#)



Languages, Node.js

Routing with React Navigation and Nesting Navigators in React Native

[Read More →](#)



Blockchain, Node.js

Ethers vs. Web3

[Read More →](#)

EngEd Author Bio



Adith Bharadwaj

Adith Bharadwaj is a senior at the National Institute of Engineering (NIE) and a Software Engineer Intern at Cisco - India. Adith has a keen interest in solving challenging problems and is a data science and machine learning enthusiast. When he's not coding, he loves drawing, working out, and watching great TV shows, movies or anime.



Deploy your apps to a supercloud in a few clicks

Try It For Free

Company

[About](#)

[Careers](#)

[Legals](#)

Resources

[Blog](#)

[Content Library](#)

[Engineering Education](#)

Support

[Docs](#)

[Community Slack](#)

[Help & Support](#)

[Release Notes](#)

[Platform Status](#)

Contact Us



 [varnish cache logo](#) [cloud native computing foundation logo](#) [the linux foundation logo](#) [if edge logo](#)

© 2022 Section

[Privacy Policy](#) [Terms of Service](#)